

W5 PRACTICE

React + Axios Integration

 *At the end of this practice, you can*

- ✓ **Apply** APIs integrations between your existing APIs with ReactJS
- ✓ **Understanding** APIs integration with using different request methods

 *Get ready before this practice!*

- ✓ **Read** the following documents to understand the nature of Express.js:
<https://expressjs.com/>
- ✓ **Read** the following documents to know more about Axios:
<https://axios-http.com/docs/intro>
- ✓ **Read** the following documents to know more about Axios with React:
<https://www.digitalocean.com/community/tutorials/react-axios-react>

 *How to submit this practice?*

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



EXERCISE 1 – APIs integration for Articles

🔗 For this exercise you will start with a **START CODE (EX-1)**

Goals

- ✓ Integrate React with RESTful APIs
- ✓ Use Axios to perform GET, POST, PUT, DELETE requests
- ✓ Display and manipulate dynamic data in components
- ✓ Apply controlled components and form handling
- ✓ Understand state, effect hooks, and API-driven routing

Context

You are a frontend developer at the same news company. Your job now is to connect the existing REST API built in Express.js to a React frontend to allow users to:

1. Browse, view, and manage articles
2. Filter articles by journalist or category
3. Create and update content via forms

Q1 – Display All Articles

Task: Build a component that fetches and displays all articles.

API Used: GET /articles

Axios REQUEST in ReactJS

```
useEffect(() => {  
  axios.get('http://localhost:5000/articles')  
    .then(res => setArticles(res.data))  
    .catch(err => console.error(err));  
}, []);
```

Q2 – View Article Details

Task: Show full content of an article using dynamic route /articles/:id.

API Used: GET /articles/:id

Use useParams() from React Router.

Axios REQUEST in ReactJS

```
axios.get(`http://localhost:5000/articles/${id}`)
```

Q3 – Add New Article

Task: Create a form to add a new article.

API Used: `POST /articles`

Fields: `title, content, journalistId, categoryId`

Axios POST REQUEST to create an article
<code>axios.post('http://localhost:5000/articles', articleData)</code>

Q4 – Update Existing Article

Task: Prefill a form and update an existing article.

API Used: `PUT /articles/:id`

Axios POST REQUEST to update an article
<code>axios.put(`http://localhost:5000/articles/\${id}`, updatedData)</code>

Reflective Questions

1. How did using `useEffect()` and `axios` help separate logic from UI?

`useEffect()` let me handle data fetching separately from the UI rendering. Instead of mixing API calls inside the JSX, I kept them in one place, which made the code cleaner. Using `axios` also simplified requests and responses, so the UI just focuses on displaying data.

2. What state challenges did you face while managing form input and API response?

The main challenge was handling multiple states that affect the same data (journalist, category, articles). Sometimes one filter would overwrite the other. I solved it by clearly tracking each state and controlling when API calls should happen.

3. How does REST structure help React developers write cleaner frontend code?

REST gives clear and predictable endpoints, so it's easy to know what to call and when. This makes the frontend logic simpler because you don't need complex data manipulation, just call the right endpoint and display the result.

EXERCISE 2 – API Integration: Filter Articles by Journalist & Category

🔑 For this exercise you will start with a **START CODE (EX-2)**

Goals

- ✓ Use dropdown selections (<select>) to trigger filtered API requests
- ✓ Understand sub-resource routes in REST (/journalists/:id/articles, etc.)
- ✓ Dynamically render filtered content
- ✓ Manage dependent state and conditional rendering in React
- ✓ Practice clean UI/UX with form inputs

Context

Your frontend application should now allow users to filter articles based on:

- **A selected journalist**
- **A selected category**

Each filter option will call a different API route and update the displayed list of articles accordingly. You will not filter client-side, but instead make API requests based on selection.

Q1 – Fetch All Journalists & Categories for Select Inputs

Task: On component mount, fetch journalists and categories to populate two dropdown menus.

APIs Used:

- GET /journalists
- GET /categories

Axios GET REQUEST to fetch an journalists and categories

```
useEffect(() => {
  axios.get('http://localhost:5000/journalists').then(res =>
    setJournalists(res.data));
  axios.get('http://localhost:5000/categories').then(res =>
    setCategories(res.data));
}, []);
```

Q2 – Filter Articles by Selected Journalist

Task: When a journalist is selected from the dropdown, fetch articles written by that journalist.

API Used:

- GET /journalists/:id/articles

Axios GET REQUEST to fetch articles

```
axios.get(`http://localhost:5000/journalists/${selectedJournalistId}/articles`)
  .then(res => setArticles(res.data));
```

Q3 – Filter Articles by Selected Category

Task: When a category is selected from the dropdown, fetch articles belonging to that category.

API Used:

Axios GET REQUEST to fetch articles
<pre>axios.get(`http://localhost:5000/journalists/\${selectedJournalistId}/articles`) .then(res => setArticles(res.data));</pre>

Q4 – (Bonus) Combine Filters (Frontend Side)

Task: Apply **combined filtering** on the frontend (e.g., articles that match both selected journalist and category) after fetching results from one of the filters.

- Adjust the backend to support combined filtering through query parameters if needed:

// Example: GET /articles?journalistId=1&categoryId=2

REFLECTIVE QUESTIONS

☛ For this part, submit it in separate PDF files

1. **How do sub-resource routes like `/journalists/:id/articles` help in designing a clear and organized API?**

Explain the benefits of using nested routes for resource relationships in REST APIs.

Sub-resource routes make the API feel more organized because they show clear relationships between data. For example, `/journalists/:id/articles` means “get articles written by this journalist.” This improves readability and helps understand how data is connected.

2. **What challenges did you face when managing multiple filter states (journalist and category) in React?**

Discuss how you handled state updates and conditional rendering when multiple inputs affect the same output.

The main challenge was handling two filters that both affect the same data. It required keeping separate states for journalist and category, then writing logic to decide which API to call. I solved this by using conditional checks in one function and making sure the UI updates correctly when filters change or reset.

3. **What would be the advantages and disadvantages of handling the filtering entirely on the frontend versus using API-based filtering?**

Compare client-side filtering (in-memory) vs. server-side filtering (via query or nested routes).

Frontend filtering is fast since data is already loaded, but it doesn't work well with large datasets. Backend filtering is more efficient and scalable because it only sends needed data.

4. **If you needed to allow filtering by both journalist and category at the same time on the backend, how would you modify the API structure?**

Think about adding query parameters or designing a new combined route.

I would use query parameters like:

`/articles?journalistId=1&categoryId=2`

This allows flexible filtering without creating too many routes.

5. **How did this exercise help you understand the interaction between React state, form controls, and RESTful API data?**

Reflect on how state changes triggered data fetching and influenced the UI rendering logic.

This exercise helped me see how user input (dropdowns) updates state, which then triggers API calls. The response updates the UI automatically. It showed how important state management and API integration are for building dynamic apps.